Per-Restaurant Revenue Forecasting — Method Documentation

This document explains, end to end, how `01_sarima_revenue_forecast.py` forecasts daily revenue for each restaurant. It's meant to be readable on its own — no need to reverse-engineer the method from the code.

---

1. Purpose

Forecast each restaurant's daily revenue 14 days ahead, with a prediction interval that's actually trustworthy — not just a number, but a number the pipeline itself has checked and is willing to stand behind, falling back to a transparent, simpler estimate when it isn't confident.

2. Why ARIMAX, and why per-restaurant

Why not a single chain-wide model? A chain-level forecast (see `04_statistical_layer/04_revenue_forecasting.py`) answers "how is the business trending overall," but it can't tell you branch X will be busy Friday — different restaurants have different scale, different age, different demand patterns. Pooling them into one series throws that away.

Why not a global panel model (e.g., gradient-boosted trees across all restaurants)? That's a real option (see the README's Possible Extensions) and would let the model borrow statistical strength across restaurants. But with only 6 restaurants, there isn't much pooling benefit yet, and a per-restaurant classical time-series model is easier to validate and explain restaurant-by-restaurant — which matters when the audience is operational staff at a specific branch, not a data science team.

Why ARIMAX and not Prophet? Prophet is a good business tool, but its uncertainty intervals come from a heuristic decomposition, not a model with a real likelihood. ARIMAX (SARIMAX with exogenous regressors) gives closed-form forecast-error variance, a testable residual diagnostic, and coefficients that mean something statistically — the right choice when the point is to *validate* the forecast, not just produce one.

Why fit 6 separate models instead of one loop? Because `groupBy().applyInPandas` distributes the 6 independent fits across Spark workers in parallel — the standard "many models" pattern for this kind of problem, and it scales to more restaurants without code changes.

3. Pipeline architecture

```text
gold_v2.daily_restaurant_performance
        │
        │  groupBy("restaurant_id")
        ▼
applyInPandas(forecast_restaurant_revenue)   ← runs once per restaurant, in parallel
        │
        ├─→ forecast rows (14 days: point + 80% interval + forecast_method)
        └─→ diagnostics row (1 per restaurant per run: order, AICc, WAPE, skill vs. naive, PASS/FAIL, status)
        │
        ▼
platinum.daily_revenue_forecast_by_restaurant   (overwritten each run)
platinum.daily_revenue_forecast_diagnostics     (appended each run — an audit trail)
```

4. Step-by-step methodology

4.1 Data preparation

- Rows are grouped and summed by `activity_date` before setting a daily frequency (`asfreq("D")`) — a defensive dedup guard, since a duplicate `(restaurant_id, activity_date)` row would otherwise break the frequency assignment or silently double-count revenue.
- **Minimum history**: restaurants with fewer than 90 days of data are skipped (`SKIPPED_INSUFFICIENT_HISTORY`), not force-fit. A seasonal model with weekly differencing needs several full seasonal cycles before its parameters mean anything.
- **Log1p transform**: the model is fit on `log(1 + revenue)`, not raw revenue. Revenue is non-negative and right-skewed; a Gaussian model on the raw scale can produce prediction intervals that dip below zero, which then need an artificial clip. Back-transforming via `expm1` is bounded below by −1, so the interval tapers smoothly toward zero instead of needing a hard clamp. See `docs/statistical_methodology.md` §4.3 for the full derivation.

4.2 Order selection

- **Differencing order (`d`)**: chosen automatically via the Augmented Dickey-Fuller test — difference until the null of a unit root is rejected, capped at `d=1`.
- **Grid search**: `(p, q, P, Q, D)` are searched over a small grid (`p,q ∈ {0,1,2}`, `P,Q,D ∈ {0,1}`), fitting a candidate SARIMAX model for each combination and keeping the one with the lowest **AICc** (finite-sample-corrected AIC — penalizes extra parameters more when history is short relative to model complexity, which matters at ~1-3 years of daily data). No restaurant is forced into another restaurant's model structure — a young, low-volume branch and an established flagship branch can and do get different orders.

4.3 Exogenous regressors

Thursday and Friday calendar dummies are included as exogenous regressors alongside the seasonal term (`seasonal_order=(P,D,Q,7)`).

Open question, documented honestly: this models weekly seasonality two ways at once — implicitly via the seasonal ARIMA component, and explicitly via the dummies. This is a known pattern ("dynamic regression"), but it isn't free — it doubles the search space (adding `D` to the grid) and risks overlapping/collinear information. **This has not yet been validated against the simpler seasonal-only model on a clean run** (see §7, Known Limitations). Treat the exogenous regressors as a hypothesis being tested, not a settled improvement.

4.4 Fitting and forecasting

- The order is selected on a training split (history minus the last 14 days).
- The final model is **refit on the full history** (train + holdout) using that selected order, so the forecast uses all available data. If the full refit fails to converge, the pipeline falls back to the training-only fit rather than losing the restaurant entirely.
- The forecast is generated 14 days ahead with an 80% prediction interval (`CI_ALPHA=0.20`), then back-transformed via `expm1` and clipped at an operational cap (§4.7).

4.5 Validation: WAPE against a seasonal-naive benchmark

Metric: WAPE (`Σ|forecast − actual| / Σactual`), computed over the 14-day holdout — not MAPE, which is unstable when actuals are near zero (a real situation here, given some branches' low-revenue days).

Benchmark: rather than judging WAPE against an arbitrary fixed cutoff, the model is compared against a **seasonal-naive forecast** — simply repeating last week's actual values. This is the cheapest defensible baseline for a weekly-seasonal series, and it's the right comparison because it asks the correct question: *does the sophisticated model actually add value over doing almost nothing*, rather than *is the model's error below some number picked without reference to how noisy this particular restaurant's demand inherently is*.

\(\\text{skill vs. naive} = \\left(1 - \\frac{\\text{WAPE}_{\\text{ARIMAX}}}{\\text{WAPE}_{\\text{naive}}}\\right) \\times 100\\%\)

A positive skill score means ARIMAX beats the naive benchmark; zero or negative means it doesn't, regardless of how "good" its absolute WAPE looks in isolation. This mirrors the spirit of MASE (Mean Absolute Scaled Error, Hyndman & Koehler 2006), which exists for exactly this reason: error metrics are only meaningful relative to how hard the series is to forecast.

An earlier version of this pipeline used a flat 25% WAPE cutoff instead. That penalized every restaurant equally regardless of its inherent noise level, and on this dataset caused every single restaurant to fall back — even ones whose residuals were genuinely white noise. The naive-benchmark comparison replaced it for exactly that reason.

4.6 Residual diagnostics

A **Ljung-Box test** (H_0: no residual autocorrelation) is run on the model's *standardized* one-step-ahead forecast errors — not raw residuals, because the Kalman filter's forecast-error variance is elevated during its early "warm-up" period, and testing raw residuals risks reading that as spurious autocorrelation rather than genuine model inadequacy. `residuals_pass = 'PASS'` if the p-value exceeds 0.05 (fails to reject the null — residuals look like white noise).

4.7 Automated fallback

A restaurant's ARIMAX forecast is used **only if both**:

1. It passes the Ljung-Box test (`p > 0.05`), **and**
2. It beats the seasonal-naive benchmark (`skill vs. naive > 0`).

If either fails, the pipeline falls back to a transparent **28-day day-of-week median ± 1.28·σ** (the 1.28 is the correct z-value for an 80% two-sided normal interval, matching `CI_ALPHA` used everywhere else). This isn't a compromise — it's a deliberate choice to show a simple, explainable number instead of a sophisticated one that failed its own validation. Every forecast row carries a `forecast_method` column (`ARIMAX` or `SEASONAL_MEDIAN_FALLBACK`) so nothing downstream — dashboard or analyst — has to guess which kind of number they're looking at.

4.8 Operational capacity cap

Forecasts (point estimate and both interval bounds) are capped at 1.25× the 99th percentile of that restaurant's historical daily revenue. This is a **business-judgment ceiling layered on top of the statistical model**, not a statistical correction — grounding the forecast in physical/operational reality (a restaurant can only serve so many covers in a day) regardless of what a purely statistical extrapolation might suggest. The 99th percentile is used instead of the raw historical maximum specifically to avoid one anomalous day setting the ceiling for the entire forecast horizon.

5. Output schema

`platinum.daily_revenue_forecast_by_restaurant` (overwritten every run — always reflects the latest forecast):

Column	Meaning
`restaurant_id`, `forecast_date`	Grain
`predicted_revenue`, `lower_ci`, `upper_ci`	Point forecast and 80% interval (from ARIMAX or the fallback)
`forecast_method`	`ARIMAX` or `SEASONAL_MEDIAN_FALLBACK` — which one produced this row
`model_run_date`	When this forecast was generated

`platinum.daily_revenue_forecast_diagnostics` (append-only — an audit trail across every run; dashboards should query the `daily_revenue_forecast_diagnostics_latest` view in `dashboards/revenue_outlook_queries.sql`, not this raw table, or they'll aggregate every historical run ever recorded):

Column	Meaning
`sarima_order`, `sarima_seasonal_order`	The AICc-selected (p,d,q) and $(P,D,Q,7)$
`aic`, `aicc`	Model fit statistics
`ljung_box_pvalue`, `residuals_pass`	Residual white-noise test and its verdict
`holdout_mape_pct`	Actually WAPE (column name kept for now — see README §17)
`naive_wape_pct`	The seasonal-naive benchmark's WAPE on the same holdout
`skill_vs_naive_pct`	The comparison that drives the fallback decision (§4.5)

Column	Meaning
`status`	`OK`, `OK_FALLBACK_SEASONAL_MEDIAN`, `SKIPPED_INSUFFICIENT_HISTORY`, or `FIT_FAILED_*`

6. How to interpret a restaurant's diagnostics

- `status = OK`, `residuals_pass = PASS`, `skill_vs_naive_pct > 0`: trust the ARIMAX forecast; the model both looks well-specified and demonstrably beats a trivial baseline.
- `status = OK_FALLBACK_SEASONAL_MEDIAN`: the number on the dashboard is a day-of-week median, not ARIMAX — check `residuals_pass` and `skill_vs_naive_pct` to see *which* condition failed, since that tells you whether the issue is model misspecification (residuals fail) or the series simply being too noisy for any model to beat "repeat last week" (skill fails despite passing residuals).
- **A high `holdout_mape_pct` alone is not necessarily alarming** for a single small restaurant's daily revenue — that's an inherently noisy, small-number process. What matters is whether it's high *relative to the naive benchmark*, which `skill_vs_naive_pct` already tells you directly.

7. Known limitations & next steps

- **Exogenous Thursday/Friday regressors have not been validated against the seasonal-only model** on a clean run (§4.3) — an ablation test (same restaurants, same run, with vs. without exog) is the natural next step before trusting this adds value.
- **Holdout validation is a single 14-day split**, not a full rolling-origin (walk-forward) backtest — cheaper to run across 6+ parallel model fits, but less rigorous than the chain-level script's methodology. Worth upgrading if compute budget allows, likely on a slower cadence than the forecast refresh itself.
- `holdout_mape_pct` **is a legacy column name** now holding a WAPE value — rename once dashboard queries referencing it are updated in lockstep.
- **The 90-day minimum history and the fallback triggers are fixed constants**, not themselves validated against what's actually achievable for this data — worth revisiting if the restaurant count or data volume grows meaningfully.

8. How to run

This runs as a scheduled Databricks Job (see `resources/zaferan_sofreh_forecast_job.yml`), not inside the continuous Lakeflow pipeline — fitting a batch of SARIMAX models is a stateful estimation step best run on its own cadence (e.g., daily), independent of how often new orders stream into Bronze/Silver/Gold.